Sisteme de operare

Laborator 4

Semestrul I 2023-2024

Vlad Olaru

Laborator 4

- controlul proceselor (continuare)
- semnale

Semnale

- *wait* obliga procesul parinte sa astepte sincron terminarea copilului
- paradigma potrivita pentru programe de sistem de tipul interpretorului de comenzi (shell)
  - asteapta tiparirea comenzii de la tastatura
  - executa comanda (creeaza proces copil cu *fork* + suprascrie imaginea copilului cu cea a executabilului)
  - asteapta terminarea executiei comenzii folosind apelul *wait*
- dar daca procesul parinte are propriul lui task, care se executa in paralel cu procesele pe care le creeaza cu *fork*?
  - ex: server TCP de retea
  - procesul parinte asteapta cereri de conexiuni folosind apelul de sistem *accept*
  - cand se deblocheaza din *accept*, creeaza un proces copil care sa trateze cererea client
  - se reintoarce in bucla apeland din nou *accept* pt a trata noi cereri de conexiune client
  - daca apeleaza *wait* pt asteptarea terminarii copiilor, parintele se blocheaza si nu mai poate accepta noi cereri de la clienti

Semnale(cont.)

- solutia Unix: notificare asincrona prin *semnale*, echivalentul software al unei intreruperi de echipament (intrerupere HW)
  - se trimit fie intre procese, fie de catre kernel catre un proces
  - generate din mai multe surse:
    - apel sistem *kill(pid, semnal)*
    - comanda *kill:* kill –INT <pid>
    - tastatura: DEL, Ctrl-C, Ctrl-Z, etc
    - anumite evenimente soft si hard generate de kernel
  - ex: terminarea unui proces notificata asincron parintelui cu SIGCHLD
    - parintele apeleaza *wait/waitpid* pentru a recupera starea terminarii procesului copil
- reactia procesului la primirea unui semnal depinde de
  - tipul semnalului
  - decizia programului de a trata sau nu evenimentul semnalat

Lista de semnale

- Linux: *signum.h* contine lista de semnale
- accesibila in program prin:

```
#include <signal.h>
```

- localizarea signum:

```
$ locate signum.h
```

Name	Description	ANSI C	POSIX.1	SVR4	4.3+BSD	Default action
SIGABRT	abnormal termination (abort)	•	•	•	•	terminate w/core
SIGALRM	time out (alarm)		•	•	•	terminate
SIGBUS	hardware fault			•	•	terminate w/core
SIGCHLD	change in status of child		job	•	•	ignore
SIGCONT	continue stopped process		job	•	•	continue/ignore
SIGEMT	hardware fault			•	•	terminate w/core
SIGFPE	arithmetic exception	•	•	•	•	terminate w/core
SIGHUP	hangup		•	•	•	terminate
SIGILL	illegal hardware instruction	•	•	•	•	terminate w/core
SIGINFO	status request from keyboard				•	ignore
SIGINT	terminal interrupt character	•	•	•	•	terminate
SIGIO	asynchronous I/O			•	•	terminate/ignore
SIGIOT	hardware fault			•	•	terminate w/core
SIGKILL	termination		•	•	•	terminate
SIGPIPE	write to pipe with no readers		•	•	•	terminate
SIGPOLL	pollable event (poll)			•		terminate
SIGPROF	profiling time alarm (setitimer)			•	•	terminate
SIGPWR	power fail/restart			•		ignore
SIGQUIT	terminal quit character		•	•	•	terminate w/core
SIGSEGV	invalid memory reference	•	•	•	•	terminate w/core
SIGSTOP	stop		job	•	•	stop process
SIGSYS	invalid system call			•	•	terminate w/core
SIGTERM	termination	•	•	•	•	terminate
SIGTRAP	hardware fault			•	•	terminate w/core
SIGTSTP	terminal stop character		job	•	•	stop process
SIGTTIN	background read from control tty		job	•	•	stop process
SIGTTOU	background write to control tty		job	•	•	stop process
SIGURG	urgent condition			•	•	ignore
SIGUSR1	user-defined signal		•	•	•	terminate
SIGUSR2	user-defined signal		•	•	•	terminate
SIGVTALRM	virtual time alarm (setitimer)			•	•	terminate
SIGWINCH	terminal window size change			•	•	ignore
SIGXCPU	CPU limit exceeded (setrlimit)			•	•	terminate w/core
SIGXFSZ	file size limit exceeded (setrlimit)			•	•	terminate w/core

Figure 10.1 Unix signals.

Posibilitati de tratare a semnalelor

1) semnalul e ignorat

- specific pt. evenimente care nu rezulta in erori/consecinte majore

2) terminarea programului (semnalul “ucide” procesul)

- valabil pt. restul evenimentelor

3) tratarea semnalului cf. indicatiei/dispozitiei programului, despre care se spune ca “prinde” semnalul

- se face cu ajutorul unui apel de sistem (*signal*/*sigaction*) care asociaza o rutina de tratare a evenimentului (handler) cu semnalul

sighandler_t signal(int signum, sighandler_t handler);

- Obs: nu orice semnal poate fi “prins” ! Ex: nici un proces nu se poate feri de semnalul SIGKILL a carui consecinta este intotdeauna terminarea procesului

Dispozitii posibile de tratare a semnalelor

1) ignorea explicita a semnalului

- se foloseste SIG_IGN ca handler

#define SIG_IGN (sighandler_t) 1

2) executa actiunea asociata implicit (de catre kernel) cu semnalul

- se foloseste SIG_DFL ca handler

#define SIG_DFL (sighandler_t) 0

3) executa codul specificat de un handler identificat de o rutina/functie din program

- in acest caz, apelul de sistem *signal/sigaction* primeste ca parametru un pointer catre o functie din program care trebuie sa respecte o anumita semnatura

typedef void (*sighandler_t)(int);

Obs: setarea unei noi dispozitii de tratare a unui semnal se face cu recuperarea vechii dispozitii (valoarea de retur a apelului *signal*)

Schema tipica de apel

```
#include <signal.h>
void (*old_handler)(int);
void handler(int)       {
                ...
}
int main(int argc, char *argv[]) {
                ....
                if((old_handler = signal(SIGINT, handler)) == SIG_ERR)
                                exit(1);
                ...
}
```

Obs: `#define SIG_ERR (sighandler_t) -1`

Observatii

- ex: asteptarea asincrona a terminarii copilului in servere de retea
- handlerul se executa in procesul parinte la intoarcerea din kernel (apelul sistem)
- cata vreme parintele e blocat (sau in general executa cod in kernel mode) nu se poate apela handlerul, pt. ca s-ar face in context kernel !
- Q: cum se apeleaza handlerul desi nu exista un apel explicit?
- A: la reintoarcerea din kernel, se “simuleaza” un apel de handler, adica:
  - se aloca pe stiva user (nu pe stiva kernel !) o noua inregistrare de activare
  - se copiaza PC-ul si SP-ul curente ale procesului
  - se pregateste inregistrarea de activare (*call stack frame*) ca si cand s-ar apela handlerul (se pune parametrul pe stiva, se aloca spatiu pentru variabilele locale, se actualizeaza SP-ul, etc)
  - se seteaza PC-ul la adresa primei instructiuni din handler
  - la revenirea in user mode, procesul va executa handlerul ca si cand l-ar fi chemat explicit
- regula generala: (1) handlerul de semnale se executa la intoarcerea din kernel in user space sau (2) dupa re-schedule (adica inainte ca procesul sa primeasca din nou procesorul)

Race conditions cu semnale

- dupa tratarea semnalului, dispozitia asociata evenimentului dispare (urmatorul semnal de acel tip va fi din nou tratat cf dispozitiei implicite)
- pt a evita situatia, se rearmeaza handlerul (la inceputul rutinei handler se apeleaza din nou *signal*)
- scenariu problematic:
  - procesul parinte care executa handlerul pierde procesorul inainte se rearmeze handlerul
  - intre timp, alt proces copil termina si se trimite un nou semnal SIGCHLD
  - pentru ca parintele nu a apucat sa rearmeze handlerul, si vechea dispozitie de tratare a SIGCHLD s-a sters, se aplica dispozitia implicita
- aceasta varianta de folosire a semnalelor s.n. *unreliable signals*

Rearmarea handlerului pt unreliable signals

```
#include <signal.h>
void handler(int)  {
        signal(SIGINT, handler);
        …
}
int main(int argc, char *argv[]) {
        ….
        signal(SIGINT, handler);
        …
}
```

Solutii posibile pt race conditions cu semnale

- 1) dispozitia de tratare a semnalului nu se reseteaza odata cu prinderea semnalului
  - consecinta: daca se genereaza semnale intruna, se cheama de fiecare data handlerul si stiva user va depasi limita alocata din cauza apelurilor de handlere imbricate
- 2) dispozitia se reseteaza, dar kernelul ignora urmatoarele semnale pana cand procesul e gata sa rearmeze handlerul (i.e, sa trateze semnalul)
  - consecinta: e posibil sa se piarda informatie, pentru ca in general procesul nu stie cate semnale urmeaza sa primeasca
- 3) solutia general acceptata recurge la blocarea/deblocarea semnalelor, in mod analog intreruperilor
  - folosind un apel de sistem specific (*sigaction*), procesul instruieste kernelul sa blocheze automat livrarea de semnale pe durata executiei handlerului
  - la terminarea handlerului, kernelul deblocheaza semnalele netrimise si le livreaza procesului
  - *reliable signals*

Apeluri sistem intrerupte

- exista apeluri de sistem lente care blocheaza procesul in kernel pt. multa vreme
- daca se genereaza un semnal si procesul a decis sa “prinda” semnalul, presupunerea e ca aparitia semnalului trebuie notificata imediat procesului, ceea ce conduce la terminarea apelului sistem lent cu un cod de eroare
- consecinta: acum nu trebuie doar cercetata valoarea de retur a apelului de sistem, ci si daca apelul sistem nu a esuat din cauza primirii unui semnal
- solutia BSD (preluata si in alte sisteme): restartarea automata a unora dintre apelurile de sistem atunci cand sunt intrerupte de semnale

Reentranta handlerului

- problema similara cu thread safety
- o serie de apeluri de biblioteca nu sunt reentrante:
  - biblioteca standard I/O, foloseste variabile statice
  - alocarea/dealocarea de memorie
  - functii de biblioteca care folosesc variabile statice gen *getpwnam* sau *strtok*
- scenariu de utilizare nereentranta cu *malloc*:
  - executia *malloc* e intrerupta de catre un semnal pe care procesul il prinde
  - daca handlerul de semnal cheama si el *malloc* => race condition
    - in implementarea interna (de ex cu liste dublu inlantuite) a algoritmilor de alocare/dealocare a blocurilor de memorie folositi de *malloc*
- solutii: (1) handlerul cheama doar functii reentrante si e reentrant vis-à-vis de variabilele globale din program, sau

(2) mascarea/blocarea semnalelor cand apeleaza functii nereentrante sau care opereaza asupra unor variabile globale accesate si de catre handler

Functii reentrante apelabile din handler

_exit	fork	pipe	stat
abort*	fstat	read	sysconf
access	getegid	rename	tcdrain
alarm	geteuid	rmdir	tcflow
cfgetispeed	getgid	setgid	tcflush
cfgetospeed	getgroups	setpgid	tcgetattr
cfsetispeed	getpgrp	setsid	tcgetpgrp
cfsetospeed	getpid	setuid	tcsendbreak
chdir	getppid	sigaction	tcsetattr
chmod	getuid	sigaddset	tcsetpgrp
chown	kill	sigdelset	time
close	link	sigemptyset	times
creat	longjmp*	sigfillset	umask
dup	lseek	sigismember	uname
dup2	mkdir	signal*	unlink
execle	mkfifo	sigpending	utime
execve	open	sigprocmask	wait
exit*	pathconf	sigsuspend	waitpid
fcntl	pause	sleep	write

Figure 10.3 Reentrant functions that may be called from a signal handler.

Kill & raise

- *kill* trimite un semnal unui proces sau unui grup de procese

  int kill(pid_t pid, int signo);

  pid > 0 semnal trimis catre procesul cu ID = pid

  pid == 0 semnal trimis catre toate procesele cu GID de proces = GID-ul de proces al procesului apelant (pt care are si drepturi de a trimite semnale)

  pid < 0 semnal trimis catre toate procesele cu GID de proces = abs(pid)

  (pt care are si drepturi de a trimite semnale)

- in general, permisiunea de a trimite un semnal unui proces revine la

  real/effective UID sender == real/effective UID receiver

- procesele *root* pot trimite semnale oricarui alt proces
- *raise* permite unui proces sa-si trimita un semnal siesi

  int raise(int signo);

Alarm

- *alarm* permite setarea unui timer care va expira dupa un timp specific in viitor

unsigned int alarm(unsigned int *seconds*);

- cand perioada expira, semnalul SIGALRM este generat
  - dispozitia implicita asociata este terminarea procesului

Obs: la *seconds* se adauga in mod normal timpul necesar procesului sa ajunga sa trateze semnalul datorita intarzierilor de planificare

- daca la apelul *alarm* exista deja setata o alarma anterioara:
  - se reseteaza valoarea alarmei la noua valoare
  - intoarce nr de secunde ramas pana la expirarea vechii alarme
- *alarm(0)* anuleaza o alarma anterioara daca nu a expirat inca (si intoarce nr de secunde ramas pana la expirare)

Pause

- *pause* suspenda executia procesului apelant pana cand acesta primeste un semnal (orice semnal)

```
int pause(void);
```

- *pause* nu se intoarce decat daca s-a executat un handler de semnal si acesta s-a intors !
  - in acest caz, *pause* intoarce -1 si errno = EINTR

Reliable signals, further reading

- signal sets
- signal masks (sigprocmask)
- pending signals (sigpending)
- sigaction
- sigsuspend